

Epidemic Spreading on Complex Networks

Technical Documentation

June 2026

Contents

1	Introduction	3
2	Architecture Overview	3
3	Project Structure	3
4	Data Model	4
4.1	Node States (Simulation Engine)	4
4.2	Network Objects	4
5	Network Generation	4
5.1	Erdős–Rényi ($G(n, p)$)	4
5.2	Barabási–Albert (Preferential Attachment)	4
5.3	Watts–Strogatz (Small-World)	5
5.4	Community (Stochastic Block Model)	5
6	How Nodes Are Connected	5
6.1	Edge Storage and Adjacency	5
6.2	Degree	5
6.3	Realized Average Degree	5
7	SIR Simulation Engine	5
7.1	Initialization Sequence	6
7.2	Timestep Loop	6
7.3	Synchronous Update Semantics	6
8	Node State Transitions	6
8.1	Vaccinated Node Behavior	6
9	Intervention and Vaccination	7
9.1	Intervention Fraction f	7
9.2	Selection Strategies	7
9.2.1	Random Vaccination	7
9.2.2	Degree-Targeted Vaccination	7
9.2.3	Betweenness-Targeted Vaccination	8
10	Centrality	8
10.1	Degree Centrality	8
10.2	Betweenness Centrality	8
11	Metrics	8
11.1	Network Statistics	8
11.2	Simulation Metrics	8
11.3	Basic Reproduction Number	8

12 Frontend and Visualization	9
12.1 Simulator Controls (UI-Exposed)	9
12.2 Visual States	9
12.3 Network Layout	9
12.4 Playback	9
13 Compare Mode and PBL	9
14 Notation Summary	10

1 Introduction

This document describes the **Epidemic-Analysis** project: an interactive Problem-Based Learning (PBL) demonstration of discrete-time **SIR epidemic spreading** on synthetic graph networks. The application is built as a single-page web app using Next.js 16, React 19, TypeScript, Zustand, D3-force, and Recharts.

The simulation runs entirely in the browser with no backend. This document covers:

- how networks are generated and how nodes are connected;
- how nodes are selected for vaccination (intervention);
- how node states change over time;
- how metrics are computed;
- how the frontend visualizes results.

2 Architecture Overview

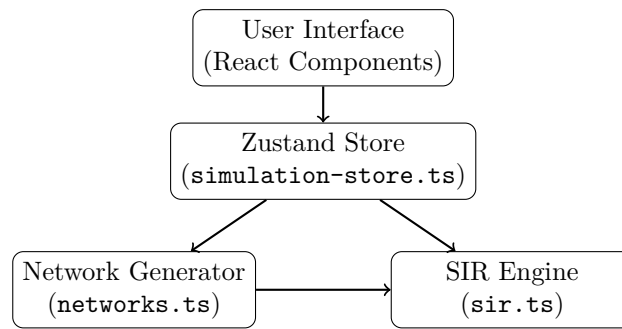


Figure 1: High-level data flow when the user runs a simulation.

Run sequence:

1. `generateNetwork(type, N, avgDegree, seed)` builds a graph.
2. A **baseline** simulation runs with no intervention.
3. An **intervention** simulation runs on the *same* network.
4. Results are stored in Zustand; the canvas renders compartment colors per timestep.

3 Project Structure

Path	Purpose
<code>src/simulation/types.ts</code>	Enums, interfaces, configuration types
<code>src/simulation/networks.ts</code>	Graph generators (ER, BA, WS, Community)
<code>src/simulation/sir.ts</code>	SIR loop, interventions, metrics
<code>src/simulation/study.ts</code>	Multi-trial batch runner (not used in UI)
<code>src/store/simulation-store.ts</code>	Global application state (Zustand)
<code>src/lib/education.ts</code>	PBL copy, compare presets, key insights
<code>src/lib/node-colors.ts</code>	Visual states, colors, compartment counting
<code>src/components/network-canvas.tsx</code>	Force-directed layout + canvas rendering

Table 1: Key source files.

4 Data Model

4.1 Node States (Simulation Engine)

The engine uses four compartments:

State	Code	Description
Susceptible	S	Can be infected
Infected	I	Can transmit and recover
Recovered	R	Immune for remainder of run
Vaccinated	V	Pre-protected; immune

Important: There is no **Exposed (E)** compartment in the engine. “Exposed” is a *visual-only* label for nodes that became infected in the current playback step.

4.2 Network Objects

Each simulation operates on a **SimNetwork**:

- **Nodes:** $\{\text{id}, \text{state}, \text{degree}\}$, where $\text{id} \in \{0, \dots, N - 1\}$.
- **Edges:** undirected pairs $\{\text{source}, \text{target}\}$.
- **Adjacency list:** $\text{adj}[i] = \text{list of neighbor IDs}$.

5 Network Generation

Entry point: `generateNetwork(type, nodeCount, avgDegree, seed)`.

All generators use the **Mulberry32** pseudo-random number generator for reproducibility.

5.1 Erdős–Rényi ($G(n, p)$)

Each unordered pair (i, j) is connected independently with probability:

$$p = \frac{\langle k \rangle}{N - 1} \quad (1)$$

where $\langle k \rangle$ is the target average degree (`avgDegree`) and N is `nodeCount`.

Properties: homogeneous degree distribution; moderate spread.

5.2 Barabási–Albert (Preferential Attachment)

Scale-free network grown by preferential attachment:

1. Start with a complete graph on $m_0 = \max(m + 1, 3)$ nodes, where $m = \max(1, \lfloor \langle k \rangle / 2 \rfloor)$.
2. Add nodes one at a time; each new node attaches m edges to existing nodes with probability proportional to their degree.

Properties: heavy-tailed degree distribution; hub nodes dominate transmission.

5.3 Watts–Strogatz (Small-World)

1. Build a ring lattice: node i connects to $(i + j) \bmod N$ for $j = 1, \dots, \lfloor k/2 \rfloor$.
2. Rewire each edge with probability $\beta_{\text{rewire}} = 0.15$ (fixed; **not** infection rate β).

Available in the engine but **not exposed** in the PBL UI.

5.4 Community (Stochastic Block Model)

Three communities with randomized sizes. For each pair (i, j) :

$$p_{\text{in}} = \frac{2\langle k \rangle}{N/3} \times 0.8 \quad (2)$$

$$p_{\text{cross}} = 0.1 \times p_{\text{in}} \quad (3)$$

Edge probability is p_{in} if same community, else p_{cross} .

Properties: dense clusters connected by sparse bridge edges.

6 How Nodes Are Connected

6.1 Edge Storage and Adjacency

Edges are undirected. After generation, `buildAdjacencyList` constructs:

$$\text{adj}[i] = \{j \mid (i, j) \in E\}$$

Each edge $\{i, j\}$ is added to both $\text{adj}[i]$ and $\text{adj}[j]$.

6.2 Degree

$$k_i = |\text{adj}[i]| \quad (4)$$

Degree is stored on each node at generation time and used for:

- degree-targeted vaccination;
- force-directed layout (repulsion and visual radius);
- network statistics and R_0 estimate.

6.3 Realized Average Degree

$$\langle k \rangle_{\text{actual}} = \frac{2|E|}{N} \quad (5)$$

This may differ slightly from the target `avgDegree`, especially for small N .

7 SIR Simulation Engine

File: `src/simulation/sir.ts`, function `runSimulation()`.

7.1 Initialization Sequence

1. All nodes $\rightarrow$ Susceptible (S)
2. `applyIntervention()` $\rightarrow$ some nodes $\rightarrow$ Vaccinated (V)
3. Seed `initialInfected` nodes from remaining S $\rightarrow$ Infected (I)
4. Record `stateHistory[0]`
5. Timestep loop (up to `maxSteps`)

7.2 Timestep Loop

For each timestep $t = 0, 1, \dots, T_{\max} - 1$:

1. Record $S(t)$, $I(t)$, $R(t)$ at the **start** of the step.
2. Stop if $I(t) = 0$.
3. For each **Infected** node i :
 - For each **Susceptible** neighbor j : infect with probability β .
 - Recover with probability γ .
4. Apply all infections, then all recoveries (synchronous update).
5. Record full state snapshot.

7.3 Synchronous Update Semantics

- A node infected in step t **cannot** transmit in the same step t .
- No within-step chaining of infections.

8 Node State Transitions

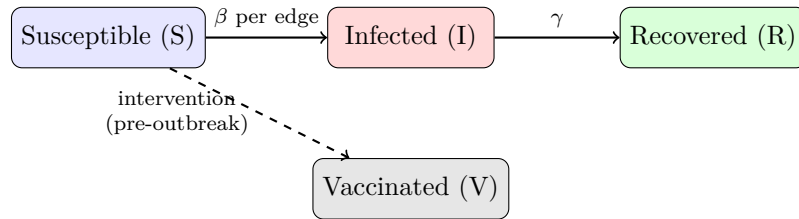


Figure 2: Node state transition diagram.

8.1 Vaccinated Node Behavior

- Vaccination occurs **before** initial infection seeding.
- Vaccinated nodes are excluded from initial infection selection.
- Only **Susceptible** neighbors can be infected during transmission.
- Vaccinated nodes never become infected or transmit.

Transition	Condition	Probability
$S \rightarrow I$	Infected neighbor; target is S	β per neighbor per step
$I \rightarrow R$	Node is I	γ per step
$S \rightarrow V$	Intervention (before outbreak)	Deterministic selection
$V \rightarrow *$	—	Never (terminal)
$R \rightarrow *$	—	Never (terminal)
$I \rightarrow S$	—	Not modeled

Table 2: State transition rules.

9 Intervention and Vaccination

9.1 Intervention Fraction f

The number of vaccinated nodes is:

$$|Q| = \text{round}(f \times N) \quad (6)$$

where `budget` = f and $N = \text{nodeCount}$.

f	$ Q $ ($N = 300$)	Meaning
0.05	15	5% vaccinated
0.10	30	10% vaccinated (default)
0.20	60	20% vaccinated

Table 3: Intervention fraction examples.

9.2 Selection Strategies

Strategy	Algorithm	UI
<code>none</code>	No vaccination	Yes
<code>random</code>	Shuffle all nodes; vaccinate first $ Q $	Yes
<code>degree-targeted</code>	Sort by degree descending; vaccinate top $ Q $ hubs	Yes (default)
<code>betweenness-targeted</code>	Sort by approximate betweenness; vaccinate top $ Q $	Engine only
<code>edge-cutting</code>	Same as betweenness (edges not actually cut)	Engine only

Table 4: Intervention strategies.

9.2.1 Random Vaccination

All nodes have equal probability of selection. Uses seeded shuffle; result is reproducible for a given `seed`.

9.2.2 Degree-Targeted Vaccination

Sort nodes by degree k_i descending; vaccinate the highest-degree hubs first.

9.2.3 Betweenness-Targeted Vaccination

Uses sampled Brandes-style BFS from $\max(10, \lfloor 0.3N \rfloor)$ source nodes. Nodes with highest betweenness scores are vaccinated first.

10 Centrality

10.1 Degree Centrality

$$C_D(i) = \frac{k_i}{N - 1} \quad (7)$$

The engine stores raw degree k_i . Since N is fixed, ranking by k_i is equivalent to ranking by $C_D(i)$.

Degree-targeted vaccination = vaccinate the top $f \times N$ nodes by $C_D(i)$.

10.2 Betweenness Centrality

Measures how often a node lies on shortest paths between other nodes. High-betweenness nodes are **bridge nodes**, especially important in community networks.

11 Metrics

11.1 Network Statistics

$$\langle k \rangle = \frac{2|E|}{N} \quad (8)$$

$$C_{\text{clustering}} = \text{mean local clustering over nodes with } k \geq 2 \quad (9)$$

11.2 Simulation Metrics

Metric	Definition
Peak infected	$\max_t I(t)$
Time to peak	First t where $I(t) = \text{peak}$
Final infected	Count of I + R + V at end
Attack rate	$\text{finalInfected}/N$
Reduction %	$(\text{final}_{\text{baseline}} - \text{final})/\text{final}_{\text{baseline}} \times 100$

11.3 Basic Reproduction Number

$$R_0 = \frac{\beta \cdot \langle k \rangle}{\gamma} \quad (10)$$

With defaults ($\beta = 0.06$, $\langle k \rangle = 4$, $\gamma = 0.04$):

$$R_0 = \frac{0.06 \times 4}{0.04} = 6.0$$

This is a mean-field approximation, not an empirical estimate from the simulation trajectory.

Parameter	Range	Default
Graph type	ER, BA, Community	Barabási–Albert
Intervention	None, Random, Degree-Targeted	Degree-Targeted
Node count N	100–500	300
β	0.02–0.08	0.06
γ	0.03–0.06	0.04
Initial infected	1–15	5

12 Frontend and Visualization

12.1 Simulator Controls (UI-Exposed)

Fixed in UI (not adjustable): avgDegree= 4, maxSteps= 80, seed= 42, budget= 0.10.

12.2 Visual States

Visual	Engine State	Color
Susceptible	S	#4d9fff
Exposed (visual only)	I (newly infected)	#f59e0b glow
Infected	I (established)	#ef4444
Recovered	R	#22c55e
Vaccinated	V	#9ca3af

A node is shown as **exposed** if it is Infected and was Susceptible in the previous snapshot.

12.3 Network Layout

D3 force simulation parameters:

- Repulsion: $-(6 + k_i \times 1.2)$
- Link distance: 18, strength: 0.15
- Collision radius: $3 + k_i \times 0.3$
- Visual node radius: $\min(3.5 + 0.4k_i, 6.5)$

12.4 Playback

Uses `requestAnimationFrame` with `msPerStep = 400/animationSpeed` (default speed $2\times \Rightarrow$ 200 ms per step).

13 Compare Mode and PBL

Three built-in compare presets:

1. **No Intervention vs Degree-Targeted** on scale-free network
2. **Random vs Scale-Free** topology (no intervention)
3. **Random vs Degree-Targeted** vaccination (both 10%)

Compare mode syncs epidemic parameters between scenarios; only topology and/or intervention differ.

14 Notation Summary

Symbol	Code variable	Default
N	<code>nodeCount</code>	300
f	<code>intervention.budget</code>	0.10
$ Q $	<code>round(budget * n)</code>	30
k_i	<code>node.degree</code>	—
$C_D(i)$	$k_i/(N - 1)$ (implicit)	—
β	<code>config.beta</code>	0.06
γ	<code>config.gamma</code>	0.04
R_0	$(\beta \langle k \rangle)/\gamma$	6.0

Table 5: Mapping between mathematical notation and code.